



EQUIPO NARANJA

ASIGNACIÓN ESCRITA IV

Uso de árboles y grafos en la solución de problemas.
Complejidad temporal de algoritmos (En ingles).

DATOS DEL EQUIPO	
Equipo	Naranja - Equipo #15
Integrantes Y Matriculas	Hugo E. Montero Fulcar Matricula 100082929
	Erasmus José Minaya Taveras Matricula 100076710
	Albert Daniel Montero Ramírez Matricula 100078731
	Luis Ángel Mora Taveras Matricula 100076432
	Fernando Enrique Mesón Acosta Matricula 100027782
Asignatura	Estructura de Datos
Institución	Universidad Abierta para Adultos (UAPA)
Fecha	5 de Junio 2026

Santiago de los Caballeros, República Dominicana
Junio 2026

Introduction

This report corresponds to Written Assignment IV of the Data Structures course, focused on the use of trees and graphs to solve problems, as well as on the analysis of the algorithmic time complexity of the implemented algorithms. The exercises developed belong to the Orange Team and were coded in C++ following the Object-Oriented Paradigm (OOP).

Four programs were developed: two based on Binary Search Trees (BST) — a prime number tree and a word tree for a search engine — and two based on undirected graphs with breadth-first traversal (BFS) — a computer network and a document distribution network between offices. Each program includes a language selector (Spanish/English) that allows the interface, menus and outputs to be displayed in both languages from the same source file.

For each function, its algorithmic complexity was determined using Big-O notation, considering the number of nodes (n) in the trees, and the number of vertices (V) and edges (E) in the graphs.

Theoretical framework

Binary Search Tree (BST)

A Binary Search Tree is a hierarchical structure in which each node has at most two children, where all values in the left subtree are smaller than the node, and all values in the right subtree are larger. This property allows insertion and search operations to have a cost proportional to the height of the tree (h): $O(\log n)$ on average when the tree is balanced, and $O(n)$ in the worst case when it degenerates into a list. The inorder traversal of a BST returns its elements in ascending order.

Graphs and BFS traversal

A graph is a set of vertices (nodes) connected by edges. In an undirected graph, edges have no direction: if A connects to B, then B also connects to A. The representation used is the adjacency list, which is memory-efficient for sparse graphs. Breadth-First Search (BFS) visits nodes level by level starting from a source, which makes it ideal for computing the minimum distance (number of hops) between two nodes and for simulating propagation or proximity-based delivery processes. Its complexity is $O(V + E)$.

Exercise 1 — Prime Number Tree (BST)

Problem description

A Binary Search Tree is built with the first 15 prime numbers. The primes are inserted in an order that produces a balanced tree, so that the height and the number of leaf nodes are representative; nevertheless, the inorder traversal always returns them in ascending order. The program displays the primes in inorder, counts the number of leaf nodes, and determines the height of the tree.

Data structure used

It uses a `Nodo` class (with an integer value and pointers to its left and right children) and an `ArbolPrimos` class that encapsulates the root and all recursive operations on the tree.

Implemented functions

- `insertar(v)`: inserts a prime while preserving the BST property.
- `mostrarInorden()`: traverses the tree in inorder and prints the sorted primes.
- `hojas()`: counts the leaf nodes (those without children).
- `altura()`: determines the height of the tree in number of levels.

Algorithmic complexity analysis (Big-O)

Function / Operation	Complexity	Justification
<code>insertar(v)</code>	$O(h) \approx O(\log n)$	Goes down one level per comparison; worst case $O(n)$.
<code>mostrarInorden()</code>	$O(n)$	Visits each of the n nodes exactly once.
<code>hojas()</code>	$O(n)$	Traverses the whole tree to count the leaves.
<code>altura()</code>	$O(n)$	Traverses the whole tree computing the maximum depth.
Overall	$O(n \log n)$	Building n insertions; queries in $O(n)$.

Source code (C++ / OOP)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Arbol de Numeros Primos (ABB) / Prime Number Tree (BST)
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
-----
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
- insertar(v)      : O(h)  -> promedio O(log n), peor caso O(n)
- mostrarInorden() : O(n)  -> visita cada nodo una vez
- hojas()          : O(n)  -> recorre todo el arbol
- altura()         : O(n)  -> recorre todo el arbol
COMPLEJIDAD GENERAL : construir el arbol = O(n log n) promedio;
                    : consultas (recorridos) = O(n)
=====*/
#include <iostream>
```

```

#include <string>
#include <algorithm>
using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Nodo del arbol / Tree node
class Nodo {
public:
    int valor;
    Nodo* izq;
    Nodo* der;
    Nodo(int v) : valor(v), izq(nullptr), der(nullptr) {}
};

// Arbol Binario de Busqueda de primos / Binary Search Tree of primes
class ArbolPrimos {
private:
    Nodo* raiz;

    // Insercion recursiva / Recursive insertion -- O(h)
    Nodo* insertar(Nodo* nodo, int v){
        if(nodo == nullptr) return new Nodo(v);
        if(v < nodo->valor) nodo->izq = insertar(nodo->izq, v);
        else if(v > nodo->valor) nodo->der = insertar(nodo->der, v);
        return nodo; // se ignoran duplicados / duplicates ignored
    }

    // Recorrido inorden (orden ascendente) / Inorder traversal -- O(n)
    void inorden(Nodo* nodo){
        if(nodo == nullptr) return;
        inorden(nodo->izq);
        cout << nodo->valor << " ";
        inorden(nodo->der);
    }

    // Conteo de hojas / Count leaf nodes -- O(n)
    int contarHojas(Nodo* nodo){
        if(nodo == nullptr) return 0;
        if(nodo->izq == nullptr && nodo->der == nullptr) return 1;
        return contarHojas(nodo->izq) + contarHojas(nodo->der);
    }

    // Altura del arbol (en numero de niveles) / Tree height -- O(n)
    int altura(Nodo* nodo){
        if(nodo == nullptr) return 0;
        return 1 + max(altura(nodo->izq), altura(nodo->der));
    }

public:
    ArbolPrimos() : raiz(nullptr) {}
    void insertar(int v){ raiz = insertar(raiz, v); }
    void mostrarInorden(){ inorden(raiz); cout << endl; }
    int hojas(){ return contarHojas(raiz); }
    int altura(){ return altura(raiz); }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    ArbolPrimos arbol;

    /* Primeros 15 numeros primos. Se insertan en un orden que produce un
       arbol balanceado, de modo que la altura y el numero de hojas sean
       representativos. El inorden siempre los devuelve ordenados.
       First 15 primes inserted in a balanced order; inorder still returns
    */
}

```

```

        them sorted. */
int primos[] = {19,7,37,3,13,29,43,2,5,11,17,23,31,41,47};
for(int p : primos) arbol.insertar(p);

cout << "\n=== " << t("ARBOL DE NUMEROS PRIMOS (ABB)",
                      "PRIME NUMBER TREE (BST)") << " ===\n";

cout << t("a) Primos en inorden: ", "a) Primes in inorder: ");
arbol.mostrarInorden();

cout << t("b) Numero de hojas: ", "b) Number of leaf nodes: ")
    << arbol.hojas() << endl;

cout << t("c) Altura del arbol (niveles): ", "c) Height of the tree (levels): ")
    << arbol.altura() << endl;

return 0;
}

```

Annex: compilation and execution

Expected output: a) inorder → 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47; b) number of leaves → 8; c) height → 4 levels. Run the program twice (option 1 = Español, option 2 = English) and capture both outputs.

[Insert the execution screenshot here]

Figure — Execution of Exercise 1 — Prime Number Tree (BST) in OnlineGDB

Exercise 2 — Word Tree for Search Engine (BST)

Problem description

A Binary Search Tree that stores keywords is implemented. The system allows inserting words, suggesting them in alphabetical order (inorder traversal), searching for exact matches, and showing how many words there are per tree level.

Data structure used

A `NodoP` class (with a 'palabra' string and pointers to its two children) and an `ArbolPalabras` class that encapsulates the root and the operations. String comparison determines the placement of each word.

Implemented functions

- `insertar(p)`: inserts a word while preserving lexicographic order.
- `sugerir()`: inorder traversal that lists the words in alphabetical order.
- `buscar(p)`: binary search for an exact match.
- `palabrasPorNivel()`: counts how many words there are on each tree level.

Algorithmic complexity analysis (Big-O)

Function / Operation	Complexity	Justification
<code>insertar(p)</code>	$O(h) \approx O(\log n)$	String comparison per level; worst case $O(n)$.
<code>sugerir()</code> [inorder]	$O(n)$	Lists the n words visiting each node once.
<code>buscar(p)</code>	$O(h) \approx O(\log n)$	Discards half of the tree at each comparison.
<code>palabrasPorNivel()</code>	$O(n)$	Traverses the tree accumulating the per-level count.
Overall	$O(n \log n)$	Insertion/search $O(\log n)$; traversals $O(n)$.

Source code (C++ / OOP)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Arbol de Palabras para Buscador (ABB) / Word Tree for Search Engine
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
-----
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
- insertar(p)      : O(h)  -> promedio O(log n), peor caso O(n)
- sugerir() [inorden] : O(n) -> lista alfabetica de todas las palabras
- buscar(p)       : O(h)  -> promedio O(log n), peor caso O(n)
- palabrasPorNivel() : O(n) -> recorre todo el arbol
COMPLEJIDAD GENERAL : insercion/busqueda = O(log n) promedio;
                    recorridos = O(n)
=====*/

#include <iostream>
#include <string>
```

```

#include <algorithm>
using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Nodo con una palabra clave / Node holding a keyword
class NodoP {
public:
    string palabra;
    NodoP* izq;
    NodoP* der;
    NodoP(string p) : palabra(p), izq(nullptr), der(nullptr) {}
};

// ABB de palabras / Word BST
class ArbolPalabras {
private:
    NodoP* raiz;

    NodoP* insertar(NodoP* n, string p){ // O(h)
        if(n == nullptr) return new NodoP(p);
        if(p < n->palabra) n->izq = insertar(n->izq, p);
        else if(p > n->palabra) n->der = insertar(n->der, p);
        return n; // duplicados ignorados / duplicates ignored
    }

    void inorden(NodoP* n){ // O(n)
        if(n == nullptr) return;
        inorden(n->izq);
        cout << n->palabra << " ";
        inorden(n->der);
    }

    bool buscar(NodoP* n, const string& p){ // O(h)
        if(n == nullptr) return false;
        if(p == n->palabra) return true;
        if(p < n->palabra) return buscar(n->izq, p);
        return buscar(n->der, p);
    }

    int altura(NodoP* n){ // O(n)
        if(n == nullptr) return 0;
        return 1 + max(altura(n->izq), altura(n->der));
    }

    void contarNivel(NodoP* n, int nivel, int conteo[]){ // O(n)
        if(n == nullptr) return;
        conteo[nivel]++;
        contarNivel(n->izq, nivel + 1, conteo);
        contarNivel(n->der, nivel + 1, conteo);
    }

public:
    ArbolPalabras() : raiz(nullptr) {}
    void insertar(string p){ raiz = insertar(raiz, p); }
    void sugerir(){ inorden(raiz); cout << endl; }
    bool buscar(string p){ return buscar(raiz, p); }

    void palabrasPorNivel(){
        int h = altura(raiz);
        if(h == 0){ cout << t("arbol vacio", "(empty tree)") << endl; return; }
        int* conteo = new int[h]();
        contarNivel(raiz, 0, conteo);
        for(int i = 0; i < h; i++){
            cout << t("Nivel ", "Level ") << i << ": "
                 << conteo[i] << t(" palabra(s)", " word(s)") << endl;
        }
        delete[] conteo;
    }
};

```

```

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    ArbolPalabras arbol;

    // Palabras clave iniciales / initial keywords
    string iniciales[] = {"computadora","algoritmo","red","datos",
                          "programa","arbol","grafo","busqueda"};
    for(string w : iniciales) arbol.insertar(w);

    int op;
    do{
        cout << "\n=== " << t("ARBOL DE PALABRAS (BUSCADOR)",
                                "WORD TREE (SEARCH ENGINE)") << " ===\n";
        cout << "1) " << t("Insertar palabra", "Insert word") << "\n";
        cout << "2) " << t("Sugerir palabras (orden alfabetico)",
                                "Suggest words (alphabetical)") << "\n";
        cout << "3) " << t("Buscar coincidencia exacta", "Search exact match") << "\n";
        cout << "4) " << t("Mostrar palabras por nivel", "Show words per level") << "\n";
        cout << "0) " << t("Salir", "Exit") << "\n> ";
        cin >> op;

        if(op == 1){
            cout << t("Palabra (sin espacios): ", "Word (no spaces): ");
            string w; cin >> w; arbol.insertar(w);
            cout << t("Palabra insertada.", "Word inserted.") << endl;
        }
        else if(op == 2){
            cout << t("Sugerencias: ", "Suggestions: ");
            arbol.sugerir();
        }
        else if(op == 3){
            cout << t("Palabra a buscar: ", "Word to search: ");
            string w; cin >> w;
            cout << (arbol.buscar(w) ? t("ENCONTRADA", "FOUND")
                                : t("NO encontrada", "NOT found")) << endl;
        }
        else if(op == 4){
            arbol.palabrasPorNivel();
        }
    } while(op != 0);

    return 0;
}

```

Annex: compilation and execution

The program loads initial words (computadora, algoritmo, red, datos, programa, arbol, grafo, busqueda). Words are read without spaces using cin to avoid buffer issues in OnlineGDB. Test the menu options and capture the screen in Spanish and English.

[Insert the execution screenshot here]

Figure — Execution of Exercise 2 — Word Tree for Search Engine (BST) in OnlineGDB

Exercise 3 — Computer Network (Undirected Graph)

Problem description

A computer network is modeled as an undirected graph. The system allows establishing connections between computers, simulating the propagation of an update through a BFS traversal, and computing the minimum number of hops (BFS distance) between two nodes.

Data structure used

A RedComputadoras class that uses an adjacency list implemented with a `map<int, vector<int>>`, where each key is a computer and its value is the list of computers connected to it.

Implemented functions

- `conectar(a, b)`: adds a bidirectional connection between two computers.
- `propagar(inicio)`: BFS traversal that shows the propagation order of the update.
- `saltosMinimos(origen, destino)`: BFS that computes the minimum distance in number of hops.

Algorithmic complexity analysis (Big-O)

Function / Operation	Complexity	Justification
<code>conectar(a, b)</code>	$O(1)$	Appends to two adjacency lists.
<code>propagar(inicio)</code>	$O(V + E)$	BFS visits each vertex and each edge once.
<code>saltosMinimos(o, d)</code>	$O(V + E)$	BFS traverses the graph until reaching the target.
Overall	$O(V + E)$	Cost dominated by the BFS traversals.

Source code (C++ / OOP)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Red de Computadoras (grafo no dirigido) / Computer Network
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
-----
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
V = numero de nodos (vertices), E = numero de aristas (conexiones)
- conectar(a,b)      : O(1)    -> insercion en lista de adyacencia
- propagar(inicio)   : O(V+E)  -> recorrido BFS completo
- saltosMinimos(o,d) : O(V+E)  -> BFS calcula distancia minima
COMPLEJIDAD GENERAL : O(V+E) por cada operacion de recorrido
=====*/

#include <iostream>
#include <vector>
#include <queue>
#include <map>
#include <string>
using namespace std;
```

```

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Grafo no dirigido con lista de adyacencia / Undirected graph (adjacency list)
class RedComputadoras {
private:
    map<int, vector<int>> ady;

public:
    // a) Conexion entre computadoras / Connection between computers -- O(1)
    void conectar(int a, int b){
        ady[a].push_back(b);
        ady[b].push_back(a); // no dirigido / undirected
    }

    // b) BFS de propagacion de actualizacion / update propagation -- O(V+E)
    void propagar(int inicio){
        map<int,bool> visit;
        queue<int> q;
        q.push(inicio); visit[inicio] = true;
        cout << t("Orden de propagacion: ", "Propagation order: ");
        while(!q.empty()){
            int u = q.front(); q.pop();
            cout << u << " ";
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; q.push(v); }
        }
        cout << endl;
    }

    // c) Saltos minimos entre dos nodos / minimum hops (BFS distance) -- O(V+E)
    int saltosMinimos(int origen, int destino){
        map<int,int> dist;
        map<int,bool> visit;
        queue<int> q;
        q.push(origen); visit[origen] = true; dist[origen] = 0;
        while(!q.empty()){
            int u = q.front(); q.pop();
            if(u == destino) return dist[u];
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; dist[v] = dist[u] + 1; q.push(v); }
        }
        return -1; // sin conexion / no path
    }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    RedComputadoras red;

    // Conexiones de ejemplo / sample connections
    red.conectar(0,1); red.conectar(0,2);
    red.conectar(1,3); red.conectar(2,3);
    red.conectar(3,4);

    int op;
    do{
        cout << "\n=== " << t("RED DE COMPUTADORAS", "COMPUTER NETWORK") << " ===\n";
        cout << "1) " << t("Conectar dos computadoras", "Connect two computers") << "\n";
        cout << "2) " << t("Propagar actualizacion (BFS)", "Propagate update (BFS)") << "\n";
        cout << "3) " << t("Saltos minimos entre dos nodos",
            "Minimum hops between two nodes") << "\n";
        cout << "0) " << t("Salir", "Exit") << "\n> ";
        cin >> op;

        if(op == 1){
            int a, b;

```

```

        cout << t("Nodo A: ", "Node A: "); cin >> a;
        cout << t("Nodo B: ", "Node B: "); cin >> b;
        red.conectar(a, b);
        cout << t("Computadoras conectadas.", "Computers connected.") << endl;
    }
    else if(op == 2){
        int s; cout << t("Nodo de inicio: ", "Start node: "); cin >> s;
        red.propagar(s);
    }
    else if(op == 3){
        int a, b;
        cout << t("Origen: ", "Source: "); cin >> a;
        cout << t("Destino: ", "Target: "); cin >> b;
        int d = red.saltosMinimos(a, b);
        if(d < 0) cout << t("No hay conexion entre los nodos.",
                           "No connection between nodes.") << endl;
        else      cout << t("Saltos minimos: ", "Minimum hops: ") << d << endl;
    }
} while(op != 0);

return 0;
}

```

Annex: compilation and execution

The program loads a sample network (0-1, 0-2, 1-3, 2-3, 3-4). Test propagation from a node and the minimum-hops calculation between two nodes. Capture the execution in both languages.

[Insert the execution screenshot here]

Figure — Execution of Exercise 3 — Computer Network in OnlineGDB

Exercise 4 — Document Distribution in an Office Network

Problem description

A company's internal document distribution network is represented as an undirected graph, where each office is a node and each direct route is an edge. The graph is built from the connections entered by the user; then the adjacency list is displayed and a BFS traversal is performed from the central office (node 0) to simulate the delivery order.

Data structure used

A RedOficinas class that uses an adjacency list implemented with a vector<vector<int>> of fixed size equal to the number of offices.

Implemented functions

- agregarConexion(a, b): registers a direct route between two offices.
- mostrarLista(): prints the complete adjacency list.
- bfsDistribucion(0): BFS traversal from the central office that generates the delivery order.

Algorithmic complexity analysis (Big-O)

Function / Operation	Complexity	Justification
agregarConexion(a, b)	$O(1)$	Inserts into two adjacency lists.
Construction (input)	$O(E)$	The E entered routes are read and registered.
mostrarLista()	$O(V + E)$	Traverses all offices and their connections.
bfsDistribucion(0)	$O(V + E)$	BFS visits each office and route only once.
Overall	$O(V + E)$	Cost dominated by construction and BFS.

Source code (C++ / OOP)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Distribucion de Documentos en Red de Oficinas
          Document Distribution in an Office Network
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
-----*/
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
V = numero de oficinas (nodos), E = numero de rutas (aristas)
- agregarConexion(a,b) : O(1)      -> insercion en lista de adyacencia
- construccion (input) : O(E)      -> se leen E rutas
- mostrarLista()      : O(V+E)     -> imprime todas las listas
- bfsDistribucion(0)  : O(V+E)     -> recorrido BFS de entrega
COMPLEJIDAD GENERAL   : O(V+E)
=====*/
#include <iostream>
#include <vector>
#include <queue>
#include <string>
```

```

using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Grafo no dirigido de oficinas / Undirected office graph
class RedOficinas {
private:
    int numOficinas;
    vector<vector<int>> ady; // lista de adyacencia / adjacency list

public:
    RedOficinas(int n) : numOficinas(n), ady(n) {}

    // a) Conexion directa entre oficinas / direct route -- O(1)
    void agregarConexion(int a, int b){
        ady[a].push_back(b);
        ady[b].push_back(a); // ruta no dirigida / undirected route
    }

    // Mostrar lista de adyacencia / display adjacency list -- O(V+E)
    void mostrarLista(){
        cout << t("\n--- Lista de adyacencia ---\n",
                  "\n--- Adjacency list ---\n");
        for(int i = 0; i < numOficinas; i++){
            cout << t("Oficina ", "Office ") << i << ": ";
            for(int v : ady[i]) cout << v << " ";
            cout << endl;
        }
    }

    // b) BFS desde la oficina central (nodo 0) / BFS from central office -- O(V+E)
    void bfsDistribucion(int inicio){
        vector<bool> visit(numOficinas, false);
        queue<int> q;
        q.push(inicio); visit[inicio] = true;
        cout << t("Orden de entrega (BFS desde oficina ",
                  "Delivery order (BFS from office ") << inicio << "): ";
        while(!q.empty()){
            int u = q.front(); q.pop();
            cout << u << " ";
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; q.push(v); }
        }
        cout << endl;
    }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    int n, m;
    cout << t("Numero de oficinas: ", "Number of offices: ");
    cin >> n;
    RedOficinas red(n);

    cout << t("Numero de conexiones (rutas): ", "Number of connections (routes): ");
    cin >> m;

    cout << t("Ingrese cada ruta como dos numeros de oficina (0 a ",
              "Enter each route as two office numbers (0 to ")
          << n - 1 << "):\n";

    for(int i = 0; i < m; i++){
        int a, b;
        cout << t("Ruta ", "Route ") << i + 1 << ": ";
        cin >> a >> b;
    }
}

```

```

        if(a >= 0 && a < n && b >= 0 && b < n){
            red.agregarConexion(a, b);
        } else {
            cout << t("Oficina invalida, intente de nuevo.",
                    "Invalid office, please try again.") << endl;
            i--; // repetir esta ruta / retry this route
        }
    }

    red.mostrarLista();
    red.bfsDistribucion(0); // oficina central = nodo 0 / central office = node 0

    return 0;
}

```

Annex: compilation and execution

Suggested test data: 5 offices, 4 routes → 0 1, 0 2, 1 3, 2 4. Result: adjacency list per office and BFS delivery order from office 0 → 0 1 2 3 4. Capture the execution in Spanish and English.

[Insert the execution screenshot here]

Figure — Execution of Exercise 4 — Document Distribution in an Office Network in OnlineGDB

Overall complexity summary

The following table consolidates the predominant time complexity of each program developed by the team:

Function / Operation	Complexity	Justification
1. Prime Number Tree	$O(n \log n) / O(n)$	BST construction and traversals.
2. Word Tree	$O(n \log n) / O(n)$	BST insertion/search and traversals.
3. Computer Network	$O(V + E)$	$O(1)$ connections and BFS traversals.
4. Office Network	$O(V + E)$	Graph construction and delivery BFS.

Conclusion

The development of these four exercises made it possible to apply binary search trees and undirected graphs to concrete problems in a practical way: sorting and counting elements, suggesting and searching for words, propagating updates across a network, and simulating the distribution of documents between offices.

The complexity analysis showed that operations on a balanced BST are efficient ($O(\log n)$ for insertion and search), while complete traversals — both in trees and in graphs — scale linearly: $O(n)$ in trees and $O(V + E)$ in graphs represented with adjacency lists. The Object-Oriented Paradigm favored encapsulation and code reuse, and the language selector made it possible to meet the requirement of presenting the interface in Spanish and English.